

Predicate locking in an extensible index framework: a supervised implementation and a question deferred

Andrey Borodin

Аннотация

Serializable snapshot isolation must detect a write that would have changed the answer of a read that already happened. The reader therefore records what it read as a predicate rather than as a set of rows, and the writer checks whether its new value falls under any recorded predicate. Where an access method offers no support, the predicate degenerates to the whole relation and every insertion conflicts with every scan.

This paper is an account of adding such support to an extensible index framework, of what the framework’s structure contributes to the argument, and of one question that was asked at the time, judged hypothetical, and turned out seven years later not to be. The implementation for the generalized search tree was carried out by Shubham Barai as a Google Summer of Code project under the present author’s supervision, and the subsequent revision for the generalized inverted index by Heikki Linnakangas; the author’s role was design review and the verification requirements described below, and the paper is written as an account rather than as a claim of priority.

We set out the covering argument that makes page granularity correct for a generalized search tree, and why the same granularity, transplanted to the inverted index, was not sound and had to be revised. We describe the requirement imposed on the project’s isolation tests — that each must fail if the access method does not implement predicate locking at all — and argue that this is the minimum standard for a test of a concurrency-control feature, since a test that passes either way measures nothing. And we return to a question raised in review in 2017: whether garbage collection may erase a page holding a predicate lock. It was correctly set aside then, because the structure did not delete pages. It began deleting them in 2019, in work by the present author, and the interaction appears not to have been addressed since.

1 Introduction

Serializable snapshot isolation [2] avoids read locks by detecting, at commit time, the dependency patterns that could yield a non-serializable schedule. One dependency it must see is between a transaction that read a range of values and a transaction that inserted a value into that range. The reader did not see the new row, and nothing in its read set records that it *would* have seen it, because the row did not exist. The mechanism is a predicate lock: the reader registers the region it examined, the writer checks whether its value falls inside any registered region.

Granularity decides whether this is affordable. With no support from the access method the region is the whole relation, and every insertion conflicts with every scan, which converts serializable isolation from a correctness guarantee into an availability problem. For a B-tree the granularity question answers itself: a scan reads a contiguous interval of a total order, and the interval is the predicate. For an extensible framework there is no order, the core does not know what a key means, and what a scan examined has no description outside the structure.

What this paper is, and whose work it reports. The implementation of predicate locking for PostgreSQL’s generalized search tree was carried out by Shubham Barai as a Google Summer of Code project in 2017, under the present author’s supervision; it was committed in 2018 with editorial changes by Teodor Sigaev and Alexander Korotkov. The subsequent revision of the corresponding mechanism for the generalized inverted index was carried out by Heikki Linnakangas. This paper is an account of

that work and of the reasoning around it, not a claim of priority over it. The author's own contributions are the design questions of Section 3, the verification requirement of Section 5, and the observation of Section 6, which is a defect rather than a feature.

Contributions.

1. The covering argument that makes page granularity correct for a generalized search tree, and the reason it does not transfer to the inverted index (Sections 2 and 4).
2. A verification requirement for concurrency-control features: each isolation test must be shown to fail when the feature is absent (Section 5).
3. A question raised in review in 2017 and correctly deferred on a premise about the rest of the system; that premise was removed in 2019, and the interaction appears still to be unhandled (Section 6).

2 Why page granularity is sound here

A scan of a generalized search tree visits a set of pages and evaluates consistent against every entry of each. What did it read? Not the entries it returned, and not the entries it examined, but the *region* described by the keys of the pages it visited: had a matching entry been anywhere in that region, the scan would have found it.

That gives the granularity. A scan takes a predicate lock on every page it examines; an insertion checks for a lock on the page where its entry will be placed. The rule is correct because of a property the structure maintains for other reasons: the key of a page covers every entry on it. If a new value would be placed on a page, it falls within that page's key, hence within a region the scan of that page examined. No insertion can land on a page and yet be outside what a scan of that page covered.

The argument is worth stating in this form because it identifies exactly what the framework contributes. Nothing here depends on what a key means; it depends only on the covering invariant, which every operator class must maintain for search to work at all. The granularity is therefore available for key types written after the mechanism was.

Implemented in 3ad55863e93, released in PostgreSQL 11.

3 Two questions asked in review

Supervision of an implementation consists largely of asking what happens in cases the implementer has not reached yet. Two such questions shaped this one, and both concern the boundary between the mechanism and the rest of the structure.

Unfinished splits. An insertion may encounter a node whose split has begun but whose downlink has not yet been inserted into the parent. Completing that split is an insertion into a page, but it is not an insertion of user data: treating it as one produces conflicts corresponding to nothing in anybody's schedule. The check must distinguish them. The same applies to index construction, where entries are placed before any reader has a stake in them.

Garbage collection. Can the maintenance pass erase a page that holds a predicate lock? At the time the answer was that it could not, because this structure never deleted pages, and the question was recorded as a matter of future compatibility. Section 6 returns to it.

4 Where the analogy failed

The inverted index stores, for each key, the list of rows containing it: an entry tree over keys whose leaves hold either an inline posting list or the root of a posting tree. Page granularity was implemented for it by analogy with the above. The analogy is precisely where the trouble lay, because the covering argument of Section 2 does not hold: a page of the entry tree is not a region of a space into which an insertion must fall, and a scan of this structure does not examine a contiguous region at all.

Two defects followed, of opposite shapes. A scan that found no matching entry recorded no predicate lock — the natural reading of “lock what you read”, and wrong, because the purpose of a predicate lock is to lock the *gap*: a query asking for a key that does not exist has read exactly the fact that it does not exist. And enabling the index’s deferred-insertion mode while transactions were running allowed subsequent insertions to bypass locks already taken on entry tree pages, so a configuration change silently weakened isolation for transactions already in progress.

The revision, by Heikki Linnakangas (0bef1c0678d, PostgreSQL 11), makes a scan always lock the metapage — representing the reading of the pending list whether or not one exists, which is what makes the mode change harmless — and always lock the entry tree page it consulted, including when it found nothing. In the other direction it removes locking that was excessive: every element of a posting tree belongs to one key value, so locking that tree’s root suffices.

A cost of the mechanism surfaced later in an unexpected place: the additional locking slowed the isolation test suite enough under instrumentation to be worth reporting [1]. Correctness machinery is paid for on every scan, not only on the schedules it exists to catch.

5 A test that cannot fail measures nothing

The requirement imposed on this work, and the part of it we would most like to see adopted elsewhere, concerns the tests rather than the code. For every patch in the series the following was checked: that the patch adds isolation tests, and that *those tests fail if the access method does not implement predicate locking at all*.

The second half is the one usually omitted. An isolation test for a concurrency-control feature typically asserts that a particular interleaving produces a serialization failure. If the feature is absent, predicate locks degenerate to relation granularity, and the interleaving produces a serialization failure anyway — for the wrong reason. The test passes in both worlds and therefore distinguishes nothing. Only by running it against a build without the feature, and requiring it to fail there, does one learn that the test is sensitive to what it claims to test.

This is the same discipline we have argued for elsewhere in the verification of structural invariants, where the analogous question is whether a check actually executes rather than whether it passes. The general form: a test is evidence only to the extent that some reachable state makes it fail, and establishing that requires constructing the state, not reasoning about it.

6 The question that stopped being hypothetical

In June 2017, reviewing the first version of the patch, the author asked whether garbage collection could erase a page holding a predicate lock, and observed that the question was then only a matter of future compatibility, since the structure never deleted pages.

It began deleting them in 2019, in work by the same author, released in PostgreSQL 12. The maintenance pass now removes an emptied leaf: it marks the page deleted and removes the downlink from its parent in the same critical section. The page’s block is not recycled until every scan that might have seen the downlink has finished, which protects a scan from reading unrelated data. It does not, however, do anything with predicate locks.

The consequence appears to be a lost conflict rather than a spurious one. A scan holds a predicate lock on page P . The entries of P are deleted by other transactions and the page is removed from the tree.

A later insertion of a value in the region that P 's key covered descends to some other page Q , and the conflict check consults the locks of Q . The lock on P is never consulted again, although the scan did examine that region. By comparison, the B-tree transfers locks from a deleted page to its right sibling for exactly this reason; the generalized search tree transfers locks on a split and not on a deletion, which is a difference of one year rather than of design.

We state this as an observation from code inspection, corroborated by the author of the deletion code, and not as a measured defect: a reproducing schedule is under construction at the time of writing, and until one exists the finding is not established. We report it here because the shape of it is the subject of this paper. The 2017 answer was correct, and it was correct *relative to a premise about the rest of the system*. Nobody revisited the premise when it changed, least of all the person who changed it.

7 Evaluation

8 Limitations

Page granularity is coarser than the predicate a query describes, and on a page whose entries span a wide region it reports conflicts a finer mechanism would not. We have not characterized how often this happens on real workloads; the argument is that the granularity is sound and computable from what the structure already maintains, not that it is tight.

The correctness argument depends on the covering property of keys and therefore on the operator class maintaining it. An operator class whose union does not produce a covering key breaks search before it breaks predicate locking, so this is not an extra assumption, but it is the point at which the framework trusts code it did not write.

Section 4 reports defects found by review and by operation rather than by systematic search, and Section 6 reports one found by reading code seven years after the question that should have prompted the reading. Neither history suggests the mechanism has been exhaustively examined.

9 Conclusion

Predicate locking is where an access method must describe what a query read rather than what it returned, and the two differ by the rows that do not exist. For a structure with a covering invariant the description is free: the key of a page soundly over-approximates what a scan of that page examined. That is what made the mechanism possible for a framework whose key types are not known in advance, and it is a property the framework maintains for other reasons.

Two things generalize past the mechanism. A test of a concurrency-control feature is evidence only if it has been shown to fail without the feature; otherwise it records that the schedule was legal, which it was in both worlds. And an answer of the form “this cannot happen, because that never occurs” is a statement about the rest of the system with a date on it. The 2017 answer here was right when given, and stopped being right two years later, at the hands of the person who gave it.

Acknowledgements

Список литературы

- [1] Andres Freund et al. GIN predicate locking slows down valgrind isolationtester runs. Discussion on the postgresql-hackers mailing list, 2018.
- [2] Dan R K Ports and Kevin Grittner. Serializable snapshot isolation in postgresql. In *Proceedings of the VLDB Endowment*, volume 5, pages 1850–1861, 2012.